

Experiment-1

~by ADEEP GUPTA

1. AIM: -

To Verify and establish a Linux environment using Kali Linux in Oracle, for programming and run basic linux commands used in C programming and File management.

2. OBJECTIVE: -

- Understand online and offline methods of accessing Linux.
- Differentiate Type-1 and Type-2 hypervisors.
- Install/import and start a Kali Linux virtual machine in Oracle VirtualBox
- Create, compile, and execute a simple C program using GCC.
- Use basic navigation, file, permission, help, and input/output redirection commands.
- Record terminal output and interpret the observed results

3. PROCEDURE: -

A) SETTING UP OF A LINUX ENVIRONMENT: -

C programming can be performed in either an online or an offline Linux environment. For this experiment, the working environment used is Kali Linux running in Oracle VirtualBox on a Windows host.

- **Online method: - Cocalc.ai**

CoCalc.ai provides a browser-based Linux workspace. A user can create or open a project, launch a Linux terminal, create a C source file, compile it with GCC, and execute the generated program without installing Linux locally.

- **Offline method: -**

1) Dual Boot: - Linux and another operating system are installed on separate disk partitions. The required operating system is selected when the computer starts.

2) Single Boot: - Linux is installed as the only operating system on the computer.

3) Bootable pen drive: - A bootable USB drive can be used to start and run or install a Linux distribution.

4) Virtual Machine Manager: - (Hypervisor)

A hypervisor is software or firmware that creates and manages virtual machines and allocates host resources such as CPU, memory, storage, and networking to them.

Type-1 (Bare-metal): Runs directly on the physical hardware without a conventional host operating system. Examples include VMware ESXi and Microsoft Hyper-V Server.

Type-2 (Hosted): Runs as an application on top of a host operating system. Examples include Oracle VirtualBox and VMware Workstation. Oracle VirtualBox is used for this experiment.

5) Installation of a VMM: - (Choosing a VMM path for windows)

Step-1: Download Oracle VirtualBox for Windows hosts.

Step-2: Download the pre-built Kali Linux virtual machine for VirtualBox.

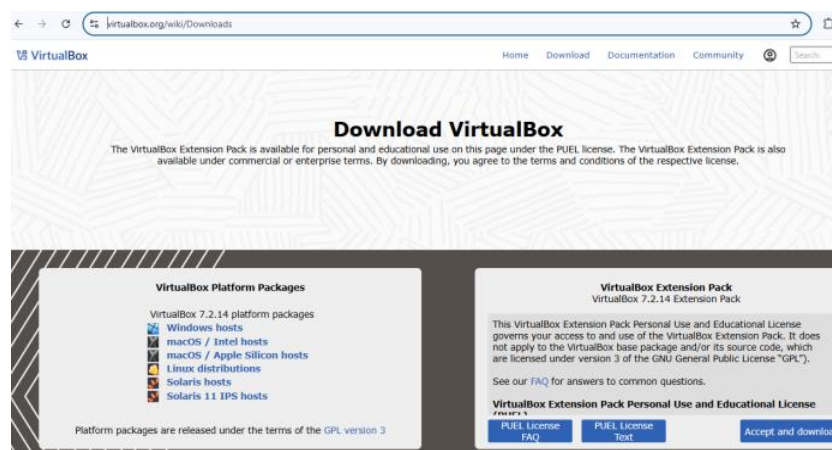
Step-3: Extract the downloaded Kali Linux virtual machine files.

Step-4: Install Oracle VirtualBox on the Windows host.

Step-5: Open VirtualBox.

Step-6: Open/import the saved Kali Linux VM.

Step-7: Start the virtual machine and log in to Kali Linux



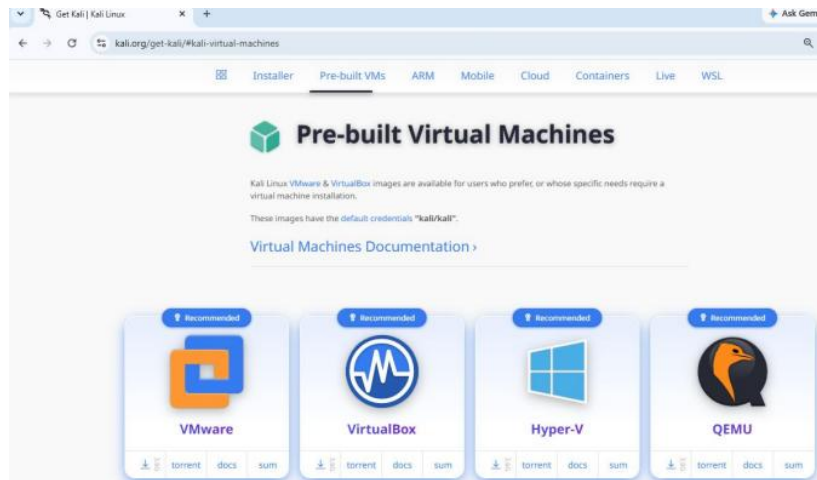


Figure 2: Pre-built Kali Linux virtual machine download options.

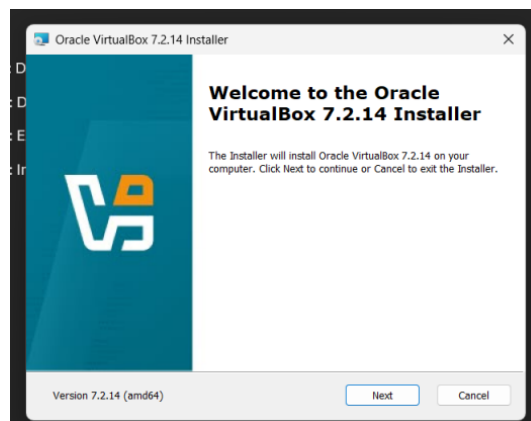


Figure 3: Oracle VirtualBox installation.

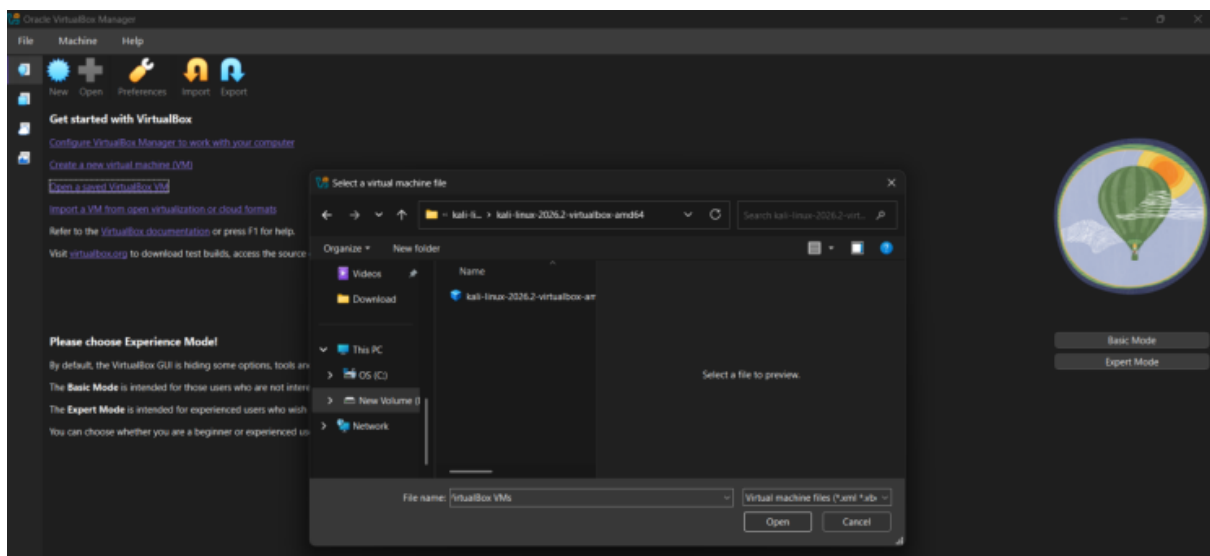


Figure 4: Opening/importing the saved Kali Linux VM in VirtualBox

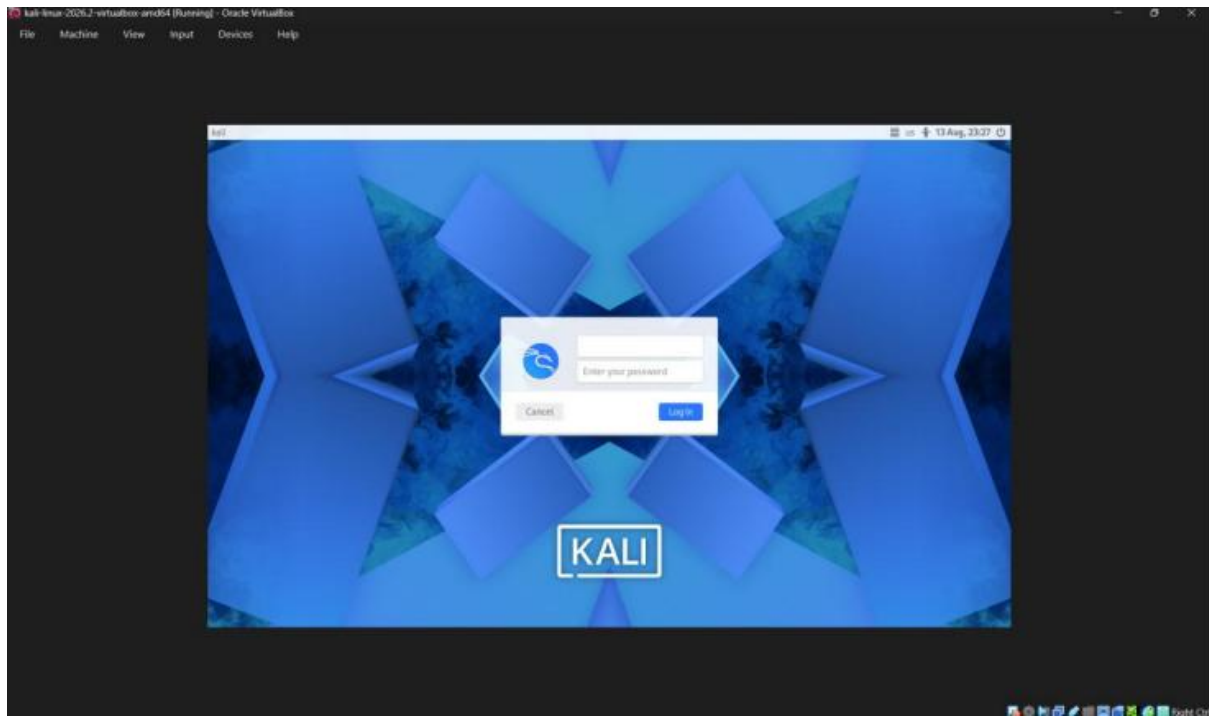


Figure 5: Kali Linux virtual machine login screen.

Thus, the Type-2 hypervisor (Oracle VirtualBox) and Kali Linux virtual machine provide the Linux programming environment required for the experiment.

Now that Linux is working Create, Compile and Execute a Simple C

```
#include <stdio.h>

int main()
{
    printf("Hello, Linux Programming Environment!\n");
    return 0;
}
```

program.

```
(kali㉿kali)-[~/Desktop/Experiment1]
$ gcc hello.c -o hello

(kali㉿kali)-[~/Desktop/Experiment1]
$ ./hello
Hello, Linux Programming Environment!

(kali㉿kali)-[~/Desktop/Experiment1]
$ ls
hello  hello.c  output.txt  sample.txt
```

Figure 6: Compiling and executing hello.c using GCC

B) Working with Linux Commands: -

The following commands were executed in the Kali Linux terminal. Output snippets are included as evidence, along with a brief explanation of each command.

Terminal Basics

Command	Brief Explanation
pwd	Displays the path of the present working directory.
mkdir	Creates a new directory.
rmdir	Removes an empty directory.
dir	Lists directory contents.
ls	Lists files and directories.
cd	Changes the current directory.
cd ..	Moves to the parent directory.
clear	Clears the terminal display.
history	Shows previously executed shell commands.
man	Opens the manual page for a command.
whoami	Displays the current user name.

File Commands and Redirection

Command	Brief Explanation
vi	Opens the vi text editor.
file	Identifies a file type.
rm	Removes a file or directory.
cp	Copies files or directories.
mv	Moves or renames files/directories.
cat	Displays or concatenates file contents.
more	Displays text one screen at a time.
less	Views text interactively with scrolling.
head	Displays the beginning of a file.
tail	Displays the end of a file.
chmod	Changes file permission bits.
>	Redirects output to a file and overwrites the file.
>>	Redirects output and appends it to a file.
<	Uses a file as standard input for a command.

4. OUTPUT SNIPPETS / EVIDENCE: -

```
(kali㉿kali)-[~/Desktop/Experiment1]
$ history
1  pwd
2  mkdir demo1
3  cd demo1
4  vi first.c
5  mkdir Demo1
6  cd demo1
7  dir
8  mkdir demo1
9  dir
10 cd demo1
11 vi second.c
12 more second.c
13 cc second.c
14 vi second.c
15 more second.c
16 cc second.c
17 ./a.out
18 vi second.c
19 cd ~/Desktop
20 mkdir Experiment2
21 cd Experiment2
22 vi circle.c
23 cat circle.c
24 gcc circle.c
25 ls
26 ./a.out
27 gcc -save-temps circle.c
28 ls
29 head -n 30 circle.i
30 head -n 30 a-circle.s
31 head
32 head -n 30 a-circle.i
33 file circle.o
34 file a-circle.o
35 file a.out
36 ./a.out
37 cd ~/Desktop
38 mkdir Experiment1
39 cd Experiment1
40 pwd
41 vi hello.c
42 gcc hello.c -o hello
43 ./hello
44 ls
45 pwd
46 whoami
```

Figure 7: Command history showing execution of terminal navigation and basic commands.

```
LS(1) User Commands LS(1)
NAME
ls - list directory contents
SYNOPSIS
ls [OPTION]... [FILE]...
DESCRIPTION
List information about the FILES (the current directory by default). Sort entries alphabetically if none of
-cftuvSUX nor --sort is specified.
Mandatory arguments to long options are mandatory for short options too.
-a, --all
do not ignore entries starting with .
-A, --almost-all
do not list implied . and ..
--author
with -l, print the author of each file
-b, --escape
print C-style escapes for nongraphic characters
--block-size=SIZE
with -l, scale sizes by SIZE when printing them; e.g., '--block-size=M'; see SIZE format below
-B, --ignore-backups
do not list implied entries ending with ~
-c
with -lt: sort by, and show, ctime (time of last change of file status information); with -l: show
ctime and sort by name; otherwise: sort by ctime, newest first
-C
list entries by columns
--color[=WHEN]
color the output WHEN; more info below
-d, --directory
list directories themselves, not their contents
-D, --dired
generate output designed for Emacs' dired mode
-f
same as -a -U
```

Figure 8: Manual page for the ls command using man ls.

```
(kali@kali)-[~/Desktop/Experiment1]
$ cd ~/Desktop/Experiment1

(kali@kali)-[~/Desktop/Experiment1]
$ vi sample.txt

(kali@kali)-[~/Desktop/Experiment1]
$ file sample.txt
sample.txt: ASCII text

(kali@kali)-[~/Desktop/Experiment1]
$ cat sample.txt
Linux is an open-sourced operating system.
This is Experiment 1.
We are learning basic Linux commands.
C programming can be performed in Linux.
Linux provides many useful file commands.
UPES Computer Science Engineering.
Tins file is created using the vi editor.
```

Figure 9: vi-created sample file verified with file and displayed using cat


```

(kali@kali)-[~/Desktop/Experiment1]
$ cp sample.txt copy.txt

(kali@kali)-[~/Desktop/Experiment1]
$ ls
copy.txt  hello  hello.c  sample.txt

(kali@kali)-[~/Desktop/Experiment1]
$ mv copy.txt renamed.txt

(kali@kali)-[~/Desktop/Experiment1]
$ ls
hello  hello.c  renamed.txt  sample.txt

(kali@kali)-[~/Desktop/Experiment1]
$ rm renamed.txt

```

Figure 10: Copying, renaming and removing files using cp, mv and rm.

```

(kali@kali)-[~/Desktop/Experiment1]
$ head sample.txt
Linux is an open-sourced operating system.
This is Experiment 1.
We are learning basic Linux commands.
C programming can be performed in Linux.
Linux provides many useful file commands.
UPES Computer Science Engineering.
Tins file is created using the vi editor.

(kali@kali)-[~/Desktop/Experiment1]
$ tail sample.txt
Linux is an open-sourced operating system.
This is Experiment 1.
We are learning basic Linux commands.
C programming can be performed in Linux.
Linux provides many useful file commands.
UPES Computer Science Engineering.
Tins file is created using the vi editor.

```

Figure 11: Viewing the beginning and end of sample.txt using head and tail.


```
Experiment2
Experiment1

Linux is an open-sourced operating system.
This is Experiment 1.
We are learning basic Linux commands.
C programming can be performed in Linux.
Linux provides many useful file commands.
UPES Computer Science Engineering.
Tins file is created using the vi editor.

sample.txt (END)
```

Figure 12: Viewing sample.txt using the more command.

```
(kali㉿kali)-[~/Desktop/Experiment1]
$ less sample.txt

(kali㉿kali)-[~/Desktop/Experiment1]
$ ls -l sample.txt
-rw-rw-r-- 1 kali kali 264 Aug 17 15:40 sample.txt

(kali㉿kali)-[~/Desktop/Experiment1]
$ chmod 755 sample.txt

(kali㉿kali)-[~/Desktop/Experiment1]
$ ls -l sample.txt
-rwxr-xr-x 1 kali kali 264 Aug 17 15:40 sample.txt
```

Figure 13: less and chmod commands; file permissions changed from rw-r--r-- to rwxr-xr-x.

```
(kali@kali)-[~/Desktop/Experiment1]
$ echo "Linux Redirection Example" > output.txt

(kali@kali)-[~/Desktop/Experiment1]
$ cat sample.txt
Linux is an open-sourced operating system.
This is Experiment 1.
We are learning basic Linux commands.
C programming can be performed in Linux.
Linux provides many useful file commands.
UPES Computer Science Engineering.
Tins file is created using the vi editor.

(kali@kali)-[~/Desktop/Experiment1]
$ echo "Linux Redirection Example" > output.txt

(kali@kali)-[~/Desktop/Experiment1]
$ cat output.txt
Linux Redirection Example
```

Figure 14: Output redirection using the > operator.

```
(kali@kali)-[~/Desktop/Experiment1]
$ echo "This line is appended." >> output.txt

(kali@kali)-[~/Desktop/Experiment1]
$ cat output.txt
Linux Redirection Example
This line is appended.

(kali@kali)-[~/Desktop/Experiment1]
$ wc -l < output.txt
2
```

Figure 15: Appending with >> and input redirection with <; wc -l reports two lines.

5. **RESULT: -**

Kali Linux programming environment was successfully set up and tested. A simple C program was written, compiled using GCC, and executed without errors. Various Linux commands were used and observed related to terminal navigation, file management, permissions, help manuals, and input/output Redirection were also practised, and their results were observed.

6. **CONCLUSION: -**

The experiment provided practical familiarity with a Linux programming environment and commonly used shell commands. It also demonstrated how a C program can be created, compiled, and executed within Kali Linux running in Oracle VirtualBox.